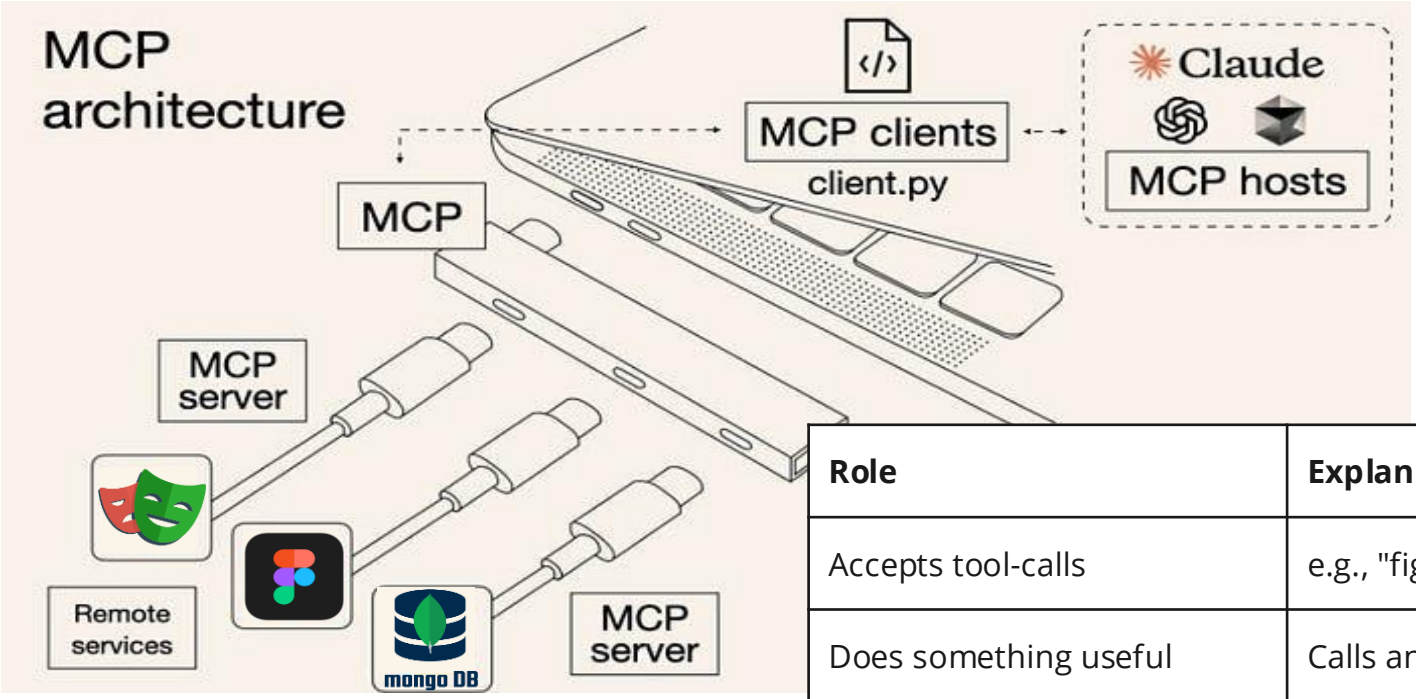
Build a working MCP Server of your own

PALO IT

tiny.cc/rni1101

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

RECAP / What *is* an MCP Server really doing?

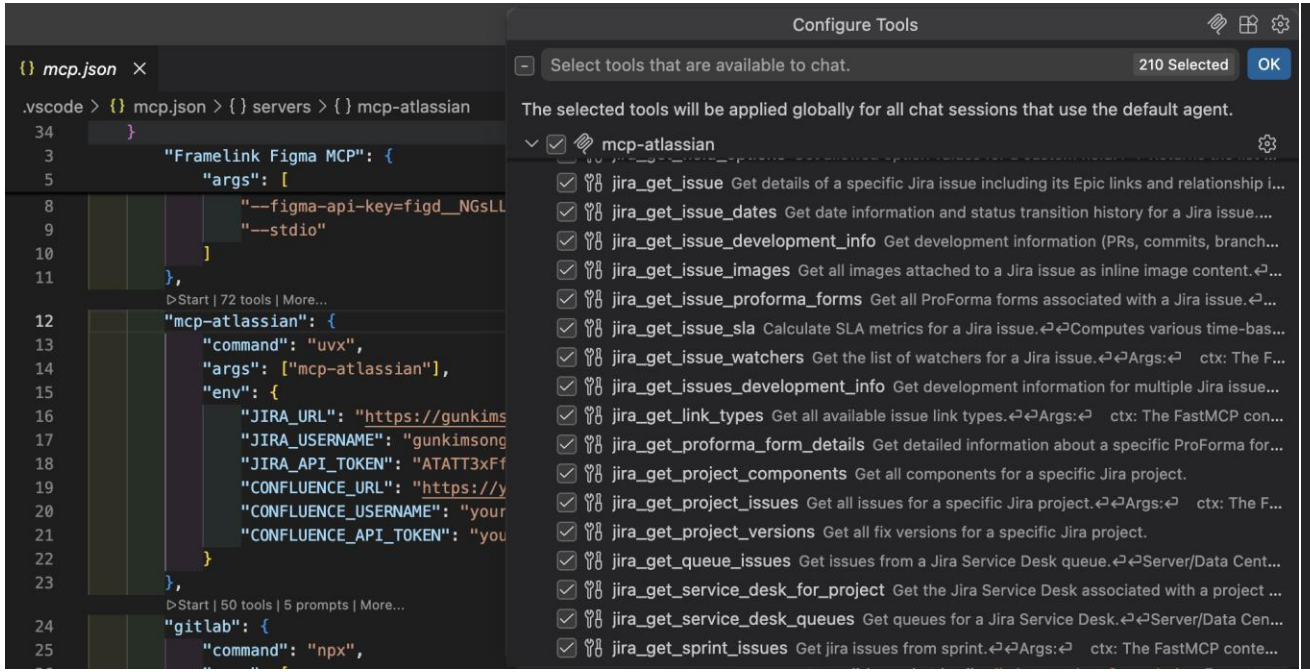


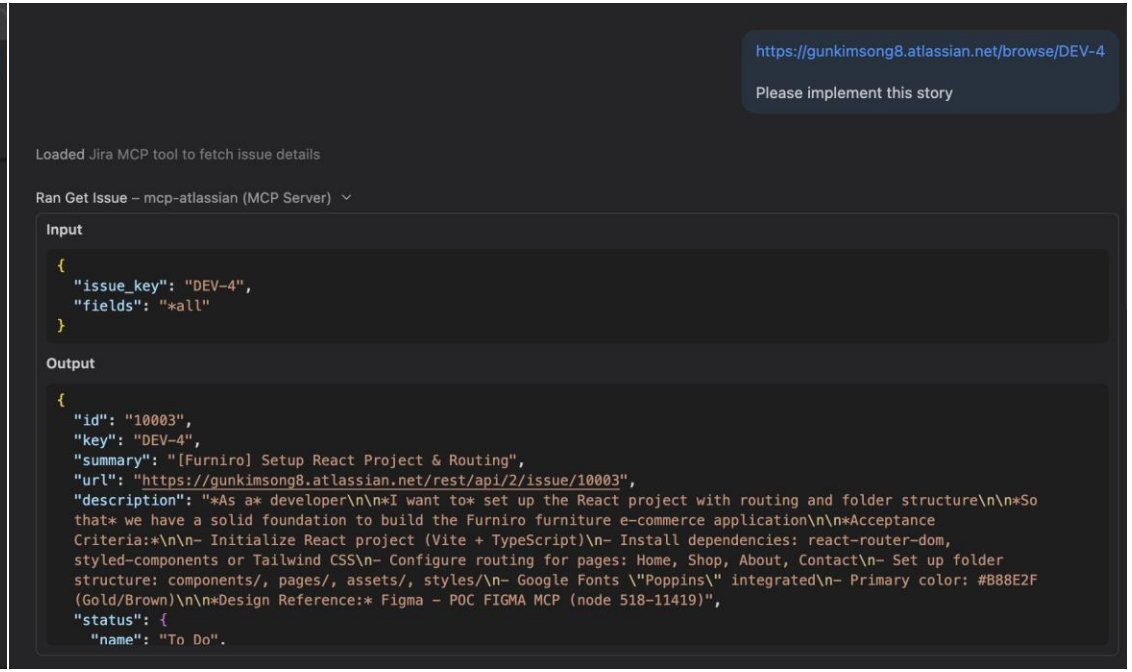
Role	Explanation
Accepts tool-calls	e.g., "figma.get_color"
Does something useful	Calls an API, reads a file, looks up a DB
Returns structured results	In JSON-RPC 2.0 format

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

BUILD YOUR OWN MCP SERVER

RECAP / What *is* an MCP Server really doing?





Accepts tool-calls

Does something useful and return In JSON-RPC 2.0 format

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

🧩 MCP Server Config Types in IDEs

```
"servers": {
    ✓Running | Stop | Restart | More...
    "atlassian": {
        "type": "http",
        "url": "https://mcp.atlassian.com/v1/mcp"
    }
}
```

```
▷Start | 2 tools | More...
"Framelink Figma MCP": {
    "command": "npx",
    "args": [
        "-y",
        "figma-developer-mcp",
        "--figma-api-key=figd__NGsLLUIsiYylWrUFK4Jzk6Nh-0f8Nznkn8EYm
        "--stdio"
    ]
},
```

Config Type	Example	How it Works	Best Use
🌐 url (remote)	"url": "http://localhost:3000"	Connects directly to an already running MCP server via HTTP	Use in dev, cloud, or CI environments
🛠 npx or command (local)	"command": "npx mcp-mobile"	Spins up a temporary MCP server via CLI at runtime	Quick local development
🐳 docker	"docker": "mobile-next/mcp"	Launches a Docker container running the MCP server	Isolated dev/test setups

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

AGENDA / OBJECTIVE

🧱 Part 1: Foundation (Concept + Mental Model)

- 🧠 Understand the message flow
  - How Copilot Agent communicates with your server
  - Key methods: tools/list, tools/call
  - Structure of a valid JSON-RPC response

🛠️ Part 2: Hands-on (Build + Run + Extend)

- 🧩 Design Your Tool Spec
  - Define tool name, description, and parameters
- 💻 Implement the Tool Logic
  - Call your service and format the response
- 🤖 Connect to Copilot
  - Setup .copilot/mcp.json
  - Test tool-call through chat prompt
- 🛠️ Debug & Validate
  - Use console logs, curl, or stdin to test edge cases
  - Handle errors gracefully in your respond() logic

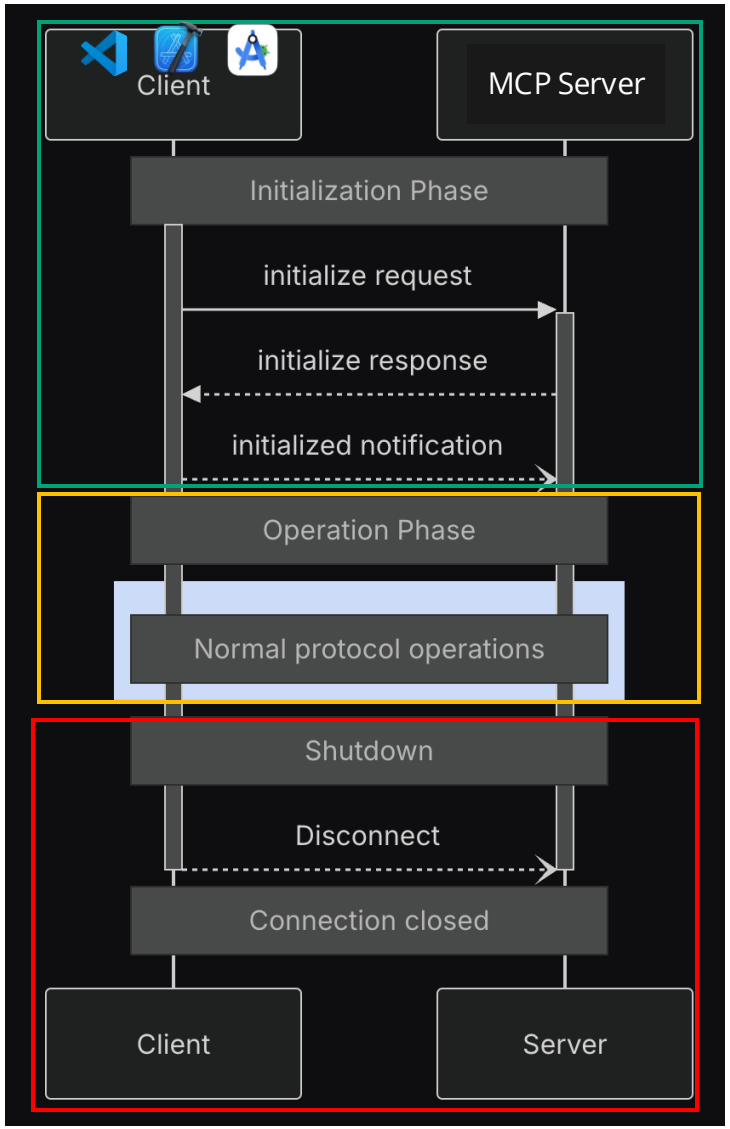
By the end of this session, you'll have a working MCP Server and the confidence to build tools your Copilot can actually use.

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

Part 1: Foundation

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

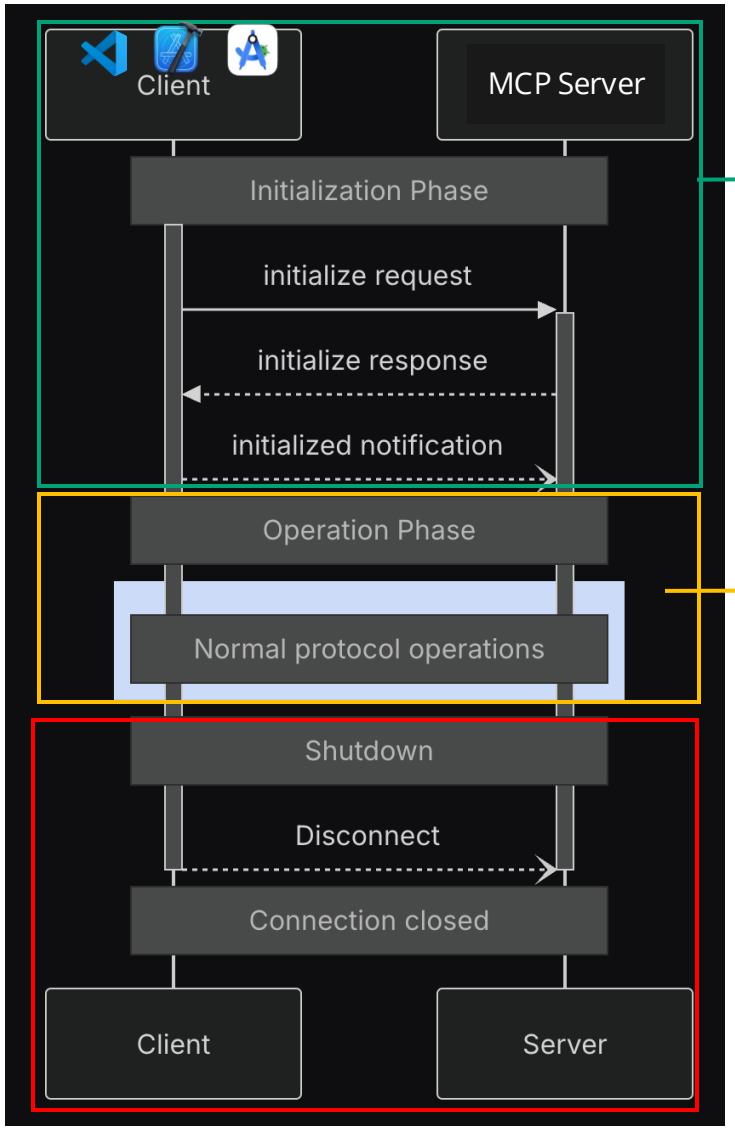
UNDERSTANDING THE ANATOMY OF MCP SERVER



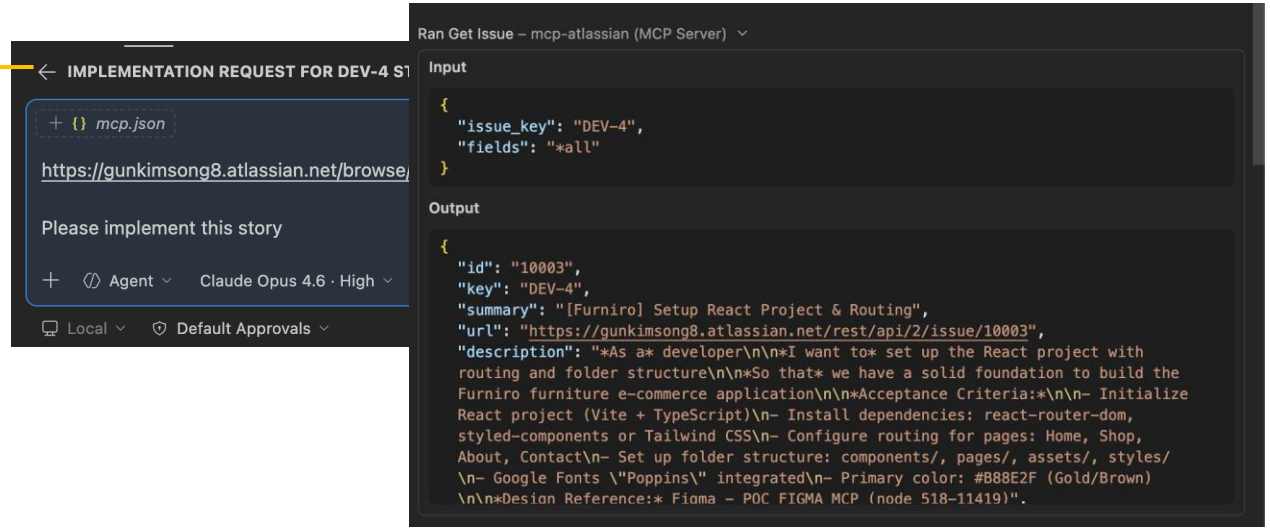
Phase	Description	What Your MCP Server Must Do
Initialization	Agent connects and sends a initialize request	Respond with a valid JSON-RPC initialize response
Initialized	Agent confirms the connection is ready by sending initialized notification	The server MUST respond with its own capabilities and information:
Operation	Agent sends actual tool requests (tools/list, tools/call, etc.)	Handle JSON-RPC methods properly (e.g., tools/list, tools/call)
Shutdown	Agent sends shutdown request to gracefully close the session	No need actions.
Disconnect / Close	Agent closes the connection completely	Clean up if necessary (e.g., stop server, clear memory, etc.)

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

UNDERSTANDING THE ANATOMY OF MCP SERVER

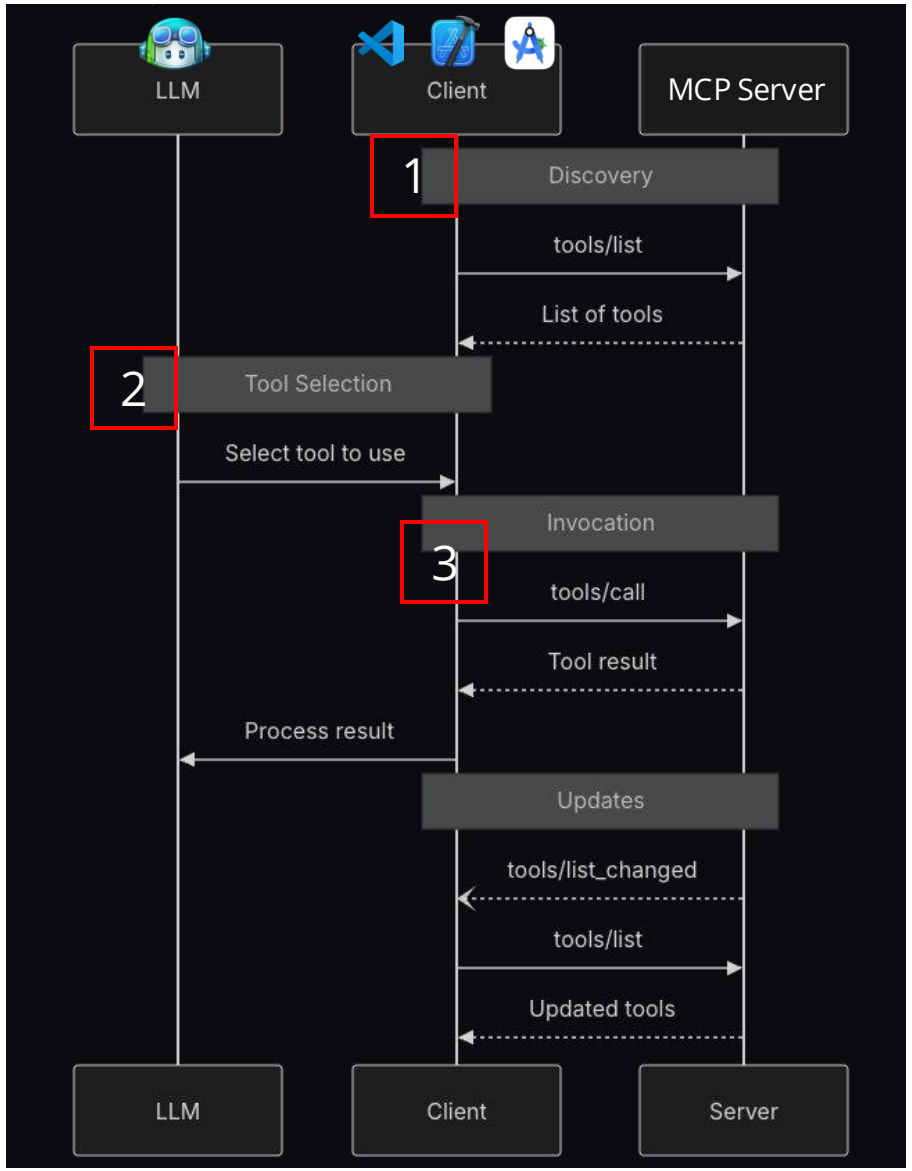



```
e > {} mcp.json > ...
{
    "servers": {
        ▷Start | 31 tools | More...
        "atlassian": {
            "type": "http",
            "url": "https://mcp.atlassian.com/v1/mcp"
        }
    }
}
```



Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

🧠 UNDERSTANDING THE OPERATION PHASE



🟣 1. Discovery – tools/list

- Client → Server: Requests a list of available tools
- Server → Client: Responds with the list of tool definitions (in JSON format)
- ✅ Your server needs to implement the tools/list method (or a route that supports it)

🎯 2. Tool Selection

- LLM (Copilot): Reads the tool metadata and decides which tool matches the user prompt
- Client → LLM: Sends selection request internally (not visible to server)

⚙️ 3. Invocation – tools/call

- Client → Server: Sends a JSON-RPC request to call the selected tool
- Server → Client: Responds with tool result
- ✅ Your MCP Server must respond to tools/call with the correct jsonrpc, id, and result

🔁 Updates – tools/list_changed

- Used when tools are dynamically added/removed
- Client → LLM: Notifies that tool list has changed
- LLM → Client → Server: Requests new tool list via tools/list again
- ✅ Optional: Only needed if your tool list changes at runtime

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

STRUCTURE OF A VALID JSON-RPC REQUESTS

```
{
  jsonrpc: "2.0";
  id: string | number;
  method: string;
  params?: {
    [key: string]: unknown;
  };
}
```

Field	Type	Required	Description
jsonrpc	string	✅ Yes	Must be "2.0" to indicate JSON-RPC 2.0 protocol version
id	string / number / null	✅ Yes for request Optional for notification	Used to match responses with requests. If null, it's a notification
method	string	✅ Yes	Name of the method to be invoked
params	object / array	Optional	Parameters to be passed to the method. Named (object) or positional (array)

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

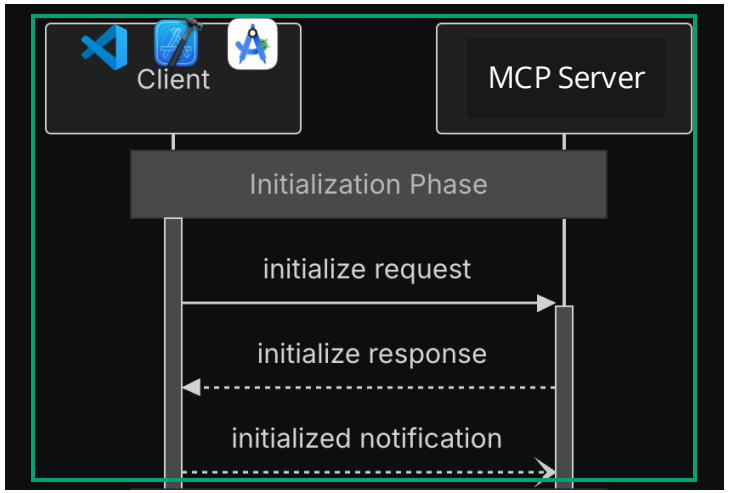
STRUCTURE OF A VALID JSON-RPC RESPONSES

```
{
  jsonrpc: "2.0";
  id: string | number;
  result?: {
    [key: string]: unknown;
  }
  error?: {
    code: number;
    message: string;
    data?: unknown;
  }
}
```

Field	Type	Required	Description
jsonrpc	string	✅ Yes	Must be "2.0" to indicate JSON-RPC 2.0 protocol version
id	string / number / null	✅ Yes for request Optional for notification	Used to match responses with requests. If null, it's a notification
method	string	✅ Yes	Name of the method to be invoked
result	any	✅ Yes (in response)	Result of a successful method invocation
error	object	✅ Yes (in error response)	Error info: must contain code, message, optional data

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

UNDERSTANDING THE ANATOMY OF MCP SERVER




```
e > {} mcp.json > ...
{
    "servers": {
        ▷Start | 31 tools | More...
        "atlassian": {
            "type": "http",
            "url": "https://mcp.atlassian.com/v1/mcp"
        }
    }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-11-25",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {},
      "elicitation": {
        "form": {},
        "url": {}
      },
      "tasks": {
        "list": {},
        "cancel": {},
        "requests": {
          "sampling": {
            "createMessage": {}
          },
          "elicitation": {
            "create": {}
          }
        }
      },
      "extensions": {
        "io.modelcontextprotocol/ui": {
          "mimeTypes": [
            "text/html;profile=mcp-app"
          ]
        }
      }
    },
    "clientInfo": {
      "name": "Visual Studio Code",
      "version": "1.113.0"
    }
  }
}
```

Field	Type	Required
jsonrpc	string	✅ Yes
id	string / number / null	✅ Yes for re Optional for notification
method	string	✅ Yes
params	object / array	Optional

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

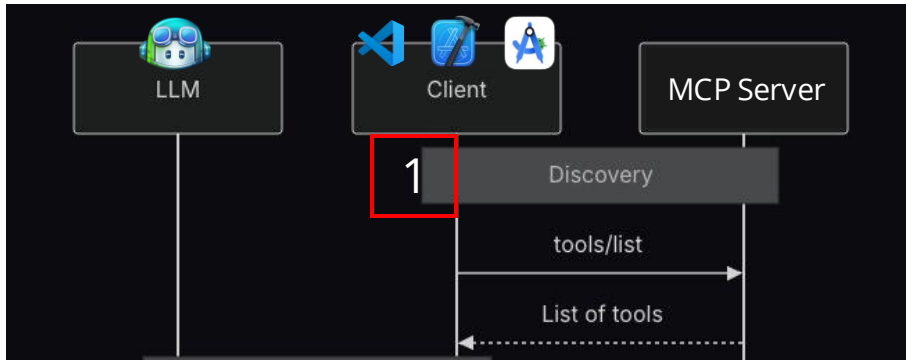
🧠 UNDERSTANDING THE ANATOMY OF MCP SERVER

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {
      "tools": {
        "listChanged": false
      }
    },
    "serverInfo": {
      "name": "weather-service",
      "version": "1.0.0"
    }
  }
}
```

Field	Type	Required
jsonrpc	string	✅ Yes
id	string / number / null	✅ Yes for request Optional for notification
method	string	✅ Yes
result	any	✅ Yes (in response)
error	object	✅ Yes (in error response)

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

🧠 UNDERSTANDING THE OPERATION PHASE



🟣 1. Discovery – tools/list

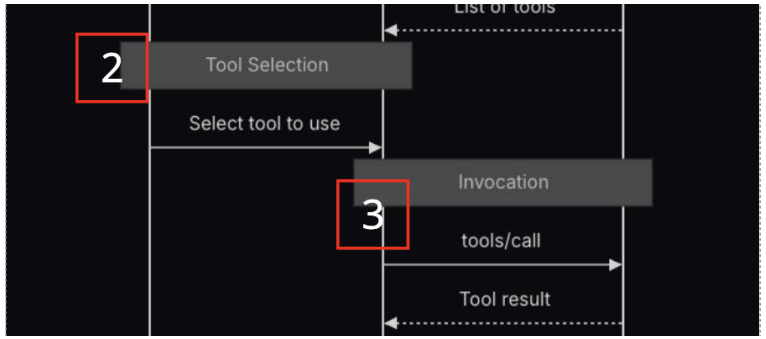
- Client → Server: Requests a list of available tools
- Server → Client: Responds with the list of tool definitions (in JSON format)
- ✅ Your server needs to implement the tools/list method (or a route that supports it)

```
🔧 MCP TOOLS/LIST: Returning 3 tools
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "tools": [
      {
        "name": "weather_get_current",
        "description": "Get current weather information for a specified city",
        "inputSchema": {
          "type": "object",
          "properties": {
            "city": {
              "type": "string",
              "description": "The name of the city to get weather for (e.g., 'bangkok', 'london', 'tokyo')"
            }
          },
          "required": [
            "city"
          ]
        }
      },
```

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

STRUCTURE OF A VALID JSON-RPC RESPONSE

To invoke a tool, clients send a tools/call request:



3. Invocation – tools/call

- Client → Server: Sends a JSON-RPC request to call the selected tool
- Server → Client: Responds with tool result
- ✅ Your MCP Server must respond to tools/call with the correctjsonrpc, id, and result

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {
      "location": "New York"
    }
  }
}
```

RESPONSE

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Current weather in New York:\nTemperature: 72°F\nConditions: Partly cloudy"
      }
    ],
    "isError": false
  }
}
```

REQUEST

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

🛠️ Part 2: Hands-on (Build + Run + Extend)

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)

FEEDBACK
Q&A

PALO IT

Information Classification: CONFIDENTIAL (sensitive business information, the level of protection is dictated by legal agreements)